

Factory Method Design Pattern

Smart Transportation Management System

Contents

1	Introduction to Design Pattern	3
2	Problem in Current System	3
3	Problems of Current Design	3
3.1	Tight Coupling	3
3.2	Poor Maintainability	3
3.3	Poor Scalability	3
3.4	Code Duplication	4
4	What is Factory Method Design Pattern?	4
5	Main Idea of Factory Pattern	4
6	Structure of Factory Method Pattern	4
7	Product Interface	4
8	Concrete Products	5
9	Factory Class	5
10	Client Code Before Refactoring	6
11	Client Code After Refactoring	6
12	Advantages of Factory Method Pattern	6
12.1	Loose Coupling	6
12.2	Easy Maintenance	6
12.3	Easy Extension	6
12.4	Cleaner Code	6
12.5	Better Scalability	6
13	Real Life Example	7
14	Future Scalability	7

15 UML Structure	7
16 Learning Outcome	7
17 Conclusion	8

1 Introduction to Design Pattern

Design Pattern হলো software development এর একটি reusable solution যা common programming problems solve করার জন্য ব্যবহার করা হয়।

এটি কোনো programming language না এবং কোনো framework ও না। বরং এটি software design করার একটি standard technique।

Factory Method Design Pattern object creation process কে organized এবং maintainable করার জন্য ব্যবহার করা হয়।

2 Problem in Current System

বর্তমান Smart Transportation Management System এ বিভিন্ন ধরনের vehicle class রয়েছে:

- Bus
- Taxi
- MotorcycleDelivery
- ElectricScooter

বর্তমান implementation এ object creation সরাসরি client code থেকে করা হচ্ছে:

```
1 Vehicle v1 = new Bus();  
2 Vehicle v2 = new Taxi();
```

এই approach এর কিছু সমস্যা রয়েছে।

3 Problems of Current Design

3.1 Tight Coupling

Client code concrete classes সম্পর্কে directly জানে।

```
1 Vehicle v = new Bus();
```

এখানে client code সরাসরি Bus class এর উপর dependent।

3.2 Poor Maintainability

নতুন vehicle type add করলে সব জায়গায় code modify করতে হয়।

3.3 Poor Scalability

Future এ যদি নতুন class add করা হয়:

- Train
- DroneDelivery
- Ambulance

তাহলে existing client code পরিবর্তন করতে হবে।

3.4 Code Duplication

অনেক জায়গায় repeated object creation logic থাকে।

4 What is Factory Method Design Pattern?

Factory Method Design Pattern হলো একটি Creational Design Pattern।

এই pattern এ object creation logic কে আলাদা একটি Factory class এর মধ্যে রাখা হয়।

Client code সরাসরি object create করে না।

বরং Factory class এর মাধ্যমে object create করা হয়।

5 Main Idea of Factory Pattern

আগে:

```
1 Vehicle v = new Bus();
```

এখন:

```
1 Vehicle v = VehicleFactory.createVehicle("bus");
```

এখন Factory internally decide করবে কোন object create হবে।

6 Structure of Factory Method Pattern

Factory Method Pattern সাধারণত নিচের components নিয়ে গঠিত:

1. Product Interface
2. Concrete Products
3. Factory Class
4. Client Code

7 Product Interface

আমাদের assignment এ Vehicle interface হলো Product Interface।

```
1 interface Vehicle {  
2  
3     void startTrip();  
4  
5     double calculateFare(double distance);  
6  
7     void assignRoute(String route);  
8  
9     void getVehicleInfo();  
10  
11     String getVehicleType();  
12 }
```

8 Concrete Products

Concrete product classes:

- Bus
- Taxi
- MotorcycleDelivery
- ElectricScooter

এগুলো সবাই Vehicle interface implement করে।

Example:

```
1 class Bus implements Vehicle {
2
3     @Override
4     public void startTrip() {
5         System.out.println("Bus trip started.");
6     }
7 }
```

9 Factory Class

Factory class object creation এর দায়িত্ব নেয়।

```
1 class VehicleFactory {
2
3     public static Vehicle createVehicle(String type) {
4
5         if(type.equalsIgnoreCase("bus")) {
6             return new Bus();
7         }
8
9         else if(type.equalsIgnoreCase("taxi")) {
10             return new Taxi();
11         }
12
13         else if(type.equalsIgnoreCase("motorcycle")) {
14             return new MotorcycleDelivery();
15         }
16
17         else if(type.equalsIgnoreCase("scooter")) {
18             return new ElectricScooter();
19         }
20
21         return null;
22     }
23 }
```

10 Client Code Before Refactoring

আগের implementation:

```
1 Vehicle vehicle = new Bus();  
2  
3 vehicle.startTrip();  
4 vehicle.assignRoute("Dhaka to Gazipur");
```

এখানে client code directly Bus class ব্যবহার করছে।

11 Client Code After Refactoring

Factory Method ব্যবহার করার পর:

```
1 Vehicle vehicle =  
2     VehicleFactory.createVehicle("bus");  
3  
4 vehicle.startTrip();  
5 vehicle.assignRoute("Dhaka to Gazipur");
```

এখন client শুধুমাত্র Factory class ব্যবহার করছে।

12 Advantages of Factory Method Pattern

12.1 Loose Coupling

Client code concrete classes সম্পর্কে জানে না।

12.2 Easy Maintenance

Object creation logic centralized থাকে।

12.3 Easy Extension

নতুন vehicle add করা সহজ।

12.4 Cleaner Code

Repeated object creation কমে যায়।

12.5 Better Scalability

Future system expansion সহজ হয়।

13 Real Life Example

Restaurant example:

Customer সরাসরি chef এর সাথে কাজ করে না।

Customer শুধু order দেয়:

“Give me a burger.”

Restaurant internally burger তৈরি করে দেয়।

ঠিক একইভাবে client শুধু বলে:

```
1 VehicleFactory.createVehicle("bus");
```

Factory internally object create করে দেয়।

14 Future Scalability

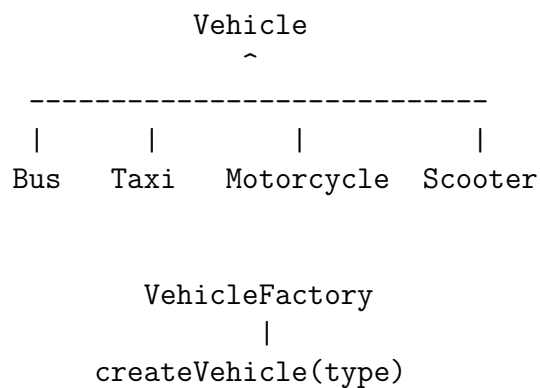
ধরুন future এ নতুন class add করা হলো:

```
1 class DroneDelivery implements Vehicle {  
2 }
```

তখন শুধুমাত্র Factory class modify করলেই হবে।

Client code change করতে হবে না।

15 UML Structure



16 Learning Outcome

এই assignment থেকে আমরা শিখতে পারি:

- Creational Design Pattern
- Factory Method Pattern
- Loose Coupling
- Better Software Architecture
- Scalable System Design

17 Conclusion

Factory Method Design Pattern object creation process কে centralized এবং maintainable করে।

Smart Transportation Management System এ এই pattern ব্যবহার করলে:

- code maintain করা সহজ হবে
- scalability বাড়বে
- future extension সহজ হবে
- tight coupling কমবে

এই কারণে বড় software systems এ Factory Method Pattern অনেক গুরুত্বপূর্ণ।